🏢 GLA UNIVERSITY — OFFICIAL 50-MCQ PRE-ASSESSMENT BANK

High Performance Computing & Heterogeneous AI Systems (GLA-HPC-BCSG701)

Day 1 Workshop Entrance & Diagnostic Pre-Assessment

Course Code: GLA-HPC-BCSG701 | **Track:** Intel & NEC Supercomputing Alliance × Edulateral Foundation **Target Audience:** 7th-Semester B.Tech Computer Science & Engineering Students (60 Students) **Lead Trainer:** Shivam Gupta | **Assessment Coordinator:** Pedina Sreeja (GLA University) **Total Questions:** Exactly 50 Multiple Choice Questions | **Duration:** 60 Minutes | **Evaluation:** +1 Mark for Correct, 0 for Unattempted **Official Reference:** `Intel & NEC - High Performance Computing.pdf` (Page 2 — Day 1 Syllabus)

📌 Pedagogical Structure & Quad-Blend Methodology

Designed to showcase the world-class teaching standard of **Edulateral Foundation** to GLA University faculty and coordinators:

1. **Numerical & Analytical Calculations (18 Questions, 36%):** Real mathematical calculations directly derived from Day 1 lectures: Theoretical Peak FLOPS (4.096 TFLOPS), Amdahl's Law speedup (5x), Gustafson's Law scaled speedup (60.85x), Gaudi 24x100GbE bandwidth (300 GB/s), DDR5 peak bandwidth (614.4 GB/s), AMX tile capacity (8 KB), AMX vs AVX-512 throughput ratio (8x), octal permissions (750), umask directory/file calculation (750/640), ext4 4KB block allocation & slack space, Bash arithmetic expansion, Linux exit code 137 (128 + Signal 9 SIGKILL), Ed25519 vs RSA key size ratio (12x smaller), 45GB scp transfer time (360s), SYCL work-group partitioning (8 groups), USM 2D matrix allocation (32 MB), MPI rank distribution (-n 64 -ppn 16), and NUMA memory latency penalty ratio (80% penalty).
2. **Applied Technical & Systems Tooling (18 Questions, 36%):** Intel oneAPI DPC++, Khronos SYCL C++, Unified Shared Memory (malloc_device, malloc_shared), Intel Level Zero, compiler flags (-O3, -xHost, -fp-model=fast=2), Intel MPI Library, and Libfabric.
3. **Logical Troubleshooting & Real-World Scenarios (8 Questions, 16%):** Pipeline silent failure trapping (set -o pipefail), SSH key permission denial (0777 -> 0600), SSH Agent forwarding socket hijacking, VFS inode link counts with open file descriptors, and NUMA cross-socket UPI latency mitigation (numactl).
4. **Foundational & Conceptual Standards (6 Questions, 12%):** Core terminology, standard acronyms (FLOPS, MIMD, VFS, Shebang, Port 22).

🔷 Module 1: Introduction to HPC

Question 01 | [Easy] [Foundational] — HPC Performance Metrics & TOP500

Which performance metric is the primary benchmark utilized to evaluate and rank the world's most powerful supercomputers on the official TOP500 list?

- **(A)** Clock frequency measured in Gigahertz (GHz)

- **(B)** Floating-point Operations Per Second (FLOPS) achieved on High Performance LINPACK (HPL)
- **(C)** Input/Output Operations Per Second (IOPS) on NVMe storage
- **(D)** Instructions Per Cycle (IPC) on integer ALU workloads

> **Correct Answer**: **(B)** `Floating-point Operations Per Second (FLOPS) achieved on High Performance LINPACK (HPL)`
> **Technical Explanation**: The TOP500 supercomputing ranking measures the maximum sustained floating-point execution rate (Rmax in FLOPS), benchmarked using High Performance LINPACK (HPL) solving a dense system of linear equations.

Question 02 | [Medium] [Numerical] — Theoretical Peak Performance (Rpeak) Calculation

A dual-socket HPC compute node is equipped with two 32-core Intel Xeon processors running at a base clock frequency of 2.0 GHz. If each core can execute 32 double-precision floating-point operations per cycle using AVX-512 vector units, what is the theoretical peak double-precision compute performance (Rpeak) of the node?

- **(A)** 1.024 TFLOPS
- **(B)** 2.048 TFLOPS
- **(C)** 8.192 TFLOPS
- **(D)** 4.096 TFLOPS

> **Correct Answer**: **(D)** `4.096 TFLOPS`
> **Technical Explanation**: Theoretical Peak Rpeak = Sockets (2) * Cores/Socket (32) * Clock (2.0 * 10^9 cycles/sec) * FLOPS/cycle (32) = 4,096 GFLOPS = 4.096 TFLOPS.

Question 03 | [Medium] [Conceptual] — The Memory Wall Bottleneck

In modern computer architecture and high-performance system design, what fundamental architectural bottleneck is termed the 'Memory Wall'?

- **(A)** The physical DRAM capacity limit imposed by 64-bit operating systems
- **(B)** The growing performance disparity where CPU processing speed grows far faster than main memory (DRAM) bandwidth and access latency
- **(C)** The maximum number of DIMM slots physically permitted on an ATX motherboard
- **(D)** The file size limit imposed by 32-bit legacy file systems

> **Correct Answer**: **(B)** `The growing performance disparity where CPU processing speed grows far faster than main memory (DRAM) bandwidth and access latency`
> **Technical Explanation**: The Memory Wall refers to the widening divergence between processor compute throughput (which has scaled rapidly via multi-core and vectorization) and DRAM access latency/bandwidth, causing cores to spend significant cycles stalled waiting for data.

Question 04 | [Medium] [Scenario] — HPC Node Roles: Login vs Compute Nodes

In a production HPC cluster environment, what is the crucial operational distinction between a Login (Head) Node and Compute Nodes?

- **(A)** Login nodes run heavy distributed deep learning training, while Compute nodes only manage user home directories
- **(B)** Login nodes are dedicated to user authentication, code compilation, and job submission, while Compute nodes are reserved exclusively for parallel workload execution managed by a scheduler
- **(C)** Login nodes contain all cluster GPUs, whereas Compute nodes feature only single-core CPUs
- **(D)** Compute nodes allow direct interactive shell access to all users, whereas Login nodes refuse SSH connections

> **Correct Answer**: **(B)** `Login nodes are dedicated to user authentication, code compilation, and job submission, while Compute nodes are reserved exclusively for parallel workload execution managed by a scheduler`
> **Technical Explanation**: Login nodes provide access for code editing, compiling, and submitting batch jobs to Slurm, while Compute nodes are isolated worker resources where heavy parallel simulations execute without interactive contention.

Question 05 | [Medium] [Numerical] — Amdahl's Law Speedup Calculation

An HPC optimization engineer profiles a simulation algorithm and finds that 20% of the runtime is strictly sequential (s = 0.20), while 80% is parallelizable (p = 0.80). According to Amdahl's Law, what is the theoretical maximum speedup limit achievable by this algorithm even if executed across an infinite number of processor cores?

- **(A)** 4x speedup
- **(B)** 5x speedup
- **(C)** 10x speedup
- **(D)** Infinite speedup

> **Correct Answer**: **(B)** `5x speedup`
> **Technical Explanation**: Amdahl's Law defines Speedup S(N) = 1 / (s + p/N). As N approaches infinity, S_max = 1 / s = 1 / 0.20 = 5x maximum achievable speedup.

Question 06 | [Hard] [Numerical] — Gustafson's Law Scaled Speedup Calculation

A researcher scales an atmospheric climate model across a 64-core HPC compute node. If 95% of the scaled execution time is spent in parallel computation (p = 0.95) and 5% is sequential (s = 0.05), what is the scaled speedup achieved according to Gustafson's Law (S(N) = s + p * N)?

- **(A)** 60.85x speedup
- **(B)** 20.00x speedup
- **(C)** 5.00x speedup
- **(D)** 128.00x speedup

> **Correct Answer**: **(A)** `60.85x speedup`
> **Technical Explanation**: Gustafson's Law formula for scaled speedup is S(N) = s + (p * N). For N=64 cores: S(64) = 0.05 + (0.95 * 64) = 0.05 + 60.80 = 60.85x speedup.

Question 07 | [Hard] [Conceptual] — Supercomputing Benchmarks: HPL vs HPCG

Why was the High Performance Conjugate Gradients (HPCG) benchmark established as a companion to the traditional High Performance LINPACK (HPL) on the TOP500 list?

- **(A)** HPL measures dense linear algebra compute throughput (high compute-to-memory ratio), whereas HPCG stresses memory bandwidth, latency, and sparse, irregular memory access patterns common in modern scientific simulations
- **(B)** HPL evaluates only CPU clock frequency, while HPCG evaluates GPU video memory display outputs
- **(C)** HPCG is designed strictly for consumer laptops, while HPL is designed only for quantum computers
- **(D)** HPL tests single-threaded Python scripts, while HPCG tests compiled C++ assembly

> **Correct Answer**: **(A)** `HPL measures dense linear algebra compute throughput (high compute-to-memory ratio), whereas HPCG stresses memory bandwidth, latency, and sparse, irregular memory access patterns common in modern scientific simulations`
> **Technical Explanation**: HPL achieves 80-90% of theoretical peak FLOPS on dense matrix math. In contrast, HPCG exercises sparse matrix-vector multiplication, unstructured grids, and low compute-to-memory traffic, typically achieving under 5% peak FLOPS, accurately reflecting real-world simulation performance.

🔷 Module 2: Intel HPC Architecture

Question 08 | [Easy] [Foundational] — Intel AMX Hardware Accelerator

What dedicated hardware matrix acceleration engine introduced in 4th Gen Intel Xeon Scalable processors (Sapphire Rapids) delivers accelerated deep learning tensor computation directly on CPU cores?

- **(A)** Intel Software Guard Extensions (Intel SGX)
- **(B)** Intel QuickAssist Technology (Intel QAT)
- **(C)** Intel Advanced Matrix Extensions (Intel AMX)
- **(D)** Intel Dynamic Execution Engine (IDEE)

> **Correct Answer**: **(C)** `Intel Advanced Matrix Extensions (Intel AMX)`
> **Technical Explanation**: Intel AMX (Advanced Matrix Extensions) is a built-in silicon accelerator on 4th and 5th Gen Xeon Scalable CPUs that accelerates matrix multiplication operations for INT8 and BF16 deep learning training and inference.

Question 09 | [Easy] [Foundational] — Intel Data Center GPU Max Microarchitecture

The Intel Data Center GPU Max Series (formerly codenamed Ponte Vecchio) engineered for exascale supercomputing and deep learning is built upon which GPU microarchitecture?

- **(A)** Intel Xe-LP (Low Power)
- **(B)** Intel Xe-HPG (High Performance Gaming)
- **(C)** Intel Gen9 Graphics
- **(D)** Intel Xe-HPC (High Performance Computing)

> **Correct Answer**: **(D)** `Intel Xe-HPC (High Performance Computing)`
> **Technical Explanation**: The Intel Data Center GPU Max Series is designed specifically for extreme supercomputing, HPC simulation, and AI training workloads using the Intel Xe-HPC microarchitecture.

Question 10 | [Medium] [Numerical] — Habana Gaudi Inter-Accelerator Scale-Out Bandwidth

A Habana Gaudi 2 AI accelerator card integrates 24 dedicated 100 Gigabit Ethernet (100GbE) RoCE v2 ports directly on the silicon for scale-out inter-chip communication. What is the total aggregate networking throughput provided per accelerator in Gigabytes per second (GB/s)?

- **(A)** 100 GB/s
- **(B)** 200 GB/s
- **(C)** 600 GB/s
- **(D)** 300 GB/s

> **Correct Answer**: **(D)** `300 GB/s`
> **Technical Explanation**: Total throughput = 24 ports * 100 Gbps = 2400 Gigabits per second. Converting to Gigabytes per second: 2400 Gbps / 8 bits/Byte = 300 GB/s aggregate interconnect bandwidth.

Question 11 | [Medium] [Numerical] — Theoretical Peak Memory Bandwidth Calculation

A dual-socket 4th Gen Intel Xeon Scalable server (Sapphire Rapids) features 8 DDR5 memory channels per socket operating at DDR5-4800 (4800 MT/s). Given a standard 64-bit (8-byte) bus width per channel, what is the theoretical peak aggregate memory bandwidth of the dual-socket system?

- **(A)** 153.6 GB/s
- **(B)** 307.2 GB/s
- **(C)** 614.4 GB/s
- **(D)** 1228.8 GB/s

> **Correct Answer**: **(C)** `614.4 GB/s`
> **Technical Explanation**: Theoretical Memory Bandwidth = Sockets (2) * Channels per socket (8) * Transfer Rate (4.8 GT/s) * Bus Width (8 Bytes) = 16 * 4.8 * 8 = 614.4 GB/s aggregate bandwidth.

Question 12 | [Medium] [Numerical] — AMX 2D Tile Register Capacity Calculation

Intel Advanced Matrix Extensions (AMX) architecture provides 8 dedicated 2D tile registers (tmm0 through tmm7). If each tile register can configure up to 16 rows of 64 bytes each, what is the maximum total data capacity stored across all 8 AMX tile registers simultaneously?

- **(A)** 512 bytes
- **(B)** 2048 bytes (2 KB)
- **(C)** 4096 bytes (4 KB)
- **(D)** 8192 bytes (8 KB)

> **Correct Answer**: **(D)** `8192 bytes (8 KB)`
> **Technical Explanation**: Each AMX tile holds 16 rows * 64 bytes = 1024 bytes (1 KB). Across all 8 tile registers (tmm0 to tmm7), the total capacity is 8 * 1024 bytes = 8192 bytes (8 KB).

Question 13 | [Hard] [Technical] — Intel Xeon CPU Max HBM Operating Modes

The Intel Xeon CPU Max Series (Sapphire Rapids with HBM2e) integrates up to 64GB of High Bandwidth Memory directly on-package. Which three memory operating modes can be configured via system BIOS?

- **(A)** Striped Mode, Mirror Mode, and Parity Mode
- **(B)** HBM-only Mode, Flat Mode (1LM), and Cache Mode (2LM)
- **(C)** Dynamic Mode, Static Mode, and Swap Mode
- **(D)** Synchronous Mode, Asynchronous Mode, and Dual-Channel Mode

> **Correct Answer**: **(B)** `HBM-only Mode, Flat Mode (1LM), and Cache Mode (2LM)`
> **Technical Explanation**: Xeon Max supports three memory modes: (1) HBM-only (runs with zero external DDR5), (2) Flat mode / 1LM (HBM and DDR5 exposed as distinct NUMA nodes), and (3) Cache mode / 2LM (HBM acts as a transparent hardware-managed L4 cache for standard DDR5 capacity).

Question 14 | [Hard] [Numerical] — Intel AMX vs AVX-512 Throughput Analysis

On a 4th Gen Xeon core, AVX-512 VNNI computes up to 256 INT8 operations per cycle, while the Intel AMX TMUL accelerator computes up to 2048 INT8 operations per cycle. By what factor (speedup ratio) does AMX outperform AVX-512 in raw INT8 matrix multiply throughput per clock cycle?

- **(A)** 8x throughput boost
- **(B)** 4x throughput boost
- **(C)** 2x throughput boost
- **(D)** 16x throughput boost

> **Correct Answer**: **(A)** `8x throughput boost`
> **Technical Explanation**: Throughput ratio = 2048 INT8 ops/cycle (AMX) / 256 INT8 ops/cycle (AVX-512) = 8.0x throughput improvement per clock cycle per core.

🔷 Module 3: Linux Basics & VFS

Question 15 | [Easy] [Foundational] — Linux Unified Filesystem Root

In the Linux Virtual File System (VFS) hierarchy, what character designates the single root directory from which all directories, files, and mounted file systems originate?

- **(A)** Backslash ('')
- **(B)** Tilde ('~')
- **(C)** Forward slash ('/')
- **(D)** Hash symbol ('#')

> **Correct Answer**: **(C)** `Forward slash ('/')`
> **Technical Explanation**: In Unix/Linux, the single forward slash '/' represents the top-level root directory of the single hierarchical filesystem tree.

Question 16 | [Easy] [Foundational] — Working Directory Navigation

Which standard Linux shell command outputs the absolute canonical pathname of the current active working directory?

- **(A)** 'cd'
- **(B)** 'pwd'
- **(C)** 'whoami'
- **(D)** 'path'

> **Correct Answer**: **(B)** `'pwd'`
> **Technical Explanation**: 'pwd' stands for 'Print Working Directory' and writes the full absolute pathname from root to the current directory.

Question 17 | [Medium] [Numerical] — Octal Permission Calculation (chmod)

In Linux octal permission notation, a systems administrator issues the command 'chmod 750 /scratch/project'. What exact read (r), write (w), and execute (x) permissions are assigned to the Owner, Group, and Others?

- **(A)** Owner: rwx (7), Group: r-x (5), Others: --- (0)
- **(B)** Owner: rw- (6), Group: rw- (6), Others: --x (1)
- **(C)** Owner: r-x (5), Group: rwx (7), Others: r-- (4)
- **(D)** Owner: rwx (7), Group: rwx (7), Others: r-x (5)

> **Correct Answer**: **(A)** `Owner: rwx (7), Group: r-x (5), Others: --- (0)`
> **Technical Explanation**: Octal values: 7 = 4(r)+2(w)+1(x) = rwx; 5 = 4(r)+0(w)+1(x) = r-x; 0 = 0(r)+0(w)+0(x) = ---. Therefore 750 grants Owner rwx, Group r-x, and Others no permissions.

Question 18 | [Medium] [Numerical] — Default File & Directory Permissions (umask)

If a Linux user's shell session has an active 'umask' setting of 027, what will be the default octal permissions assigned to a newly created directory (base 777) and a newly created regular text file (base 666) respectively?

- **(A)** Directory: 755 ('rwxr-xr-x'), File: 644 ('rw-r--r--')
- **(B)** Directory: 700 ('rwx------'), File: 600 ('rw-------')
- **(C)** Directory: 770 ('rwxrwx---'), File: 660 ('rw-rw----')
- **(D)** Directory: 750 ('rwxr-x---'), File: 640 ('rw-r-----')

> **Correct Answer**: **(D)** `Directory: 750 ('rwxr-x---'), File: 640 ('rw-r-----')`
> **Technical Explanation**: Base permissions: Directory = 777, File = 666. Applying umask 027 (bitwise bit clear): Directory = 777 - 027 = 750 ('rwxr-x---'). File = 666 masked by 027 = 640 ('rw-r-----'). Group gets r-x (dir) or r-- (file); others get zero rights.

Question 19 | [Medium] [Numerical] — Filesystem Block Allocation & Internal Fragmentation

In a Linux ext4 filesystem configured with the standard 4096-byte (4 KB) allocation block size, if a simulation outputs a configuration file of exactly 10,000 bytes, how many filesystem blocks are allocated on disk and how much internal fragmentation (slack space) is unused in the final block?

- **(A)** 2 blocks allocated, 0 bytes slack space
- **(B)** 3 blocks allocated, 2288 bytes slack space
- **(C)** 4 blocks allocated, 6384 bytes slack space
- **(D)** 1 block allocated, 5904 bytes slack space

> **Correct Answer**: **(B)** `3 blocks allocated, 2288 bytes slack space`
> **Technical Explanation**: Blocks allocated = ceiling(10,000 / 4096) = 3 blocks (total disk space = 3 * 4096 = 12,288 bytes). Internal fragmentation (slack space) = 12,288 - 10,000 = 2,288 bytes.

Question 20 | [Hard] [Technical] — Virtual Filesystems: /proc vs /sys

What is the fundamental architectural distinction between the virtual filesystems '/proc' and '/sys' in the Linux kernel?

- **(A)** '/proc' provides a view of process execution state and kernel runtime statistics (procfs), while '/sys' exposes the kernel's unified device model, hardware buses, and driver topology (sysfs)
- **(B)** '/proc' is stored permanently on an NVMe SSD, while '/sys' exists only in CPU cache
- **(C)** '/proc' is read-only for root, while '/sys' can be modified by any unprivileged user
- **(D)** '/proc' manages network routing tables, while '/sys' handles user password hashing

> **Correct Answer**: **(A)** `'/proc' provides a view of process execution state and kernel runtime statistics (procfs), while '/sys' exposes the kernel's unified device model, hardware buses, and driver topology (sysfs)`
> **Technical Explanation**: '/proc' (procfs) exposes active process metadata and runtime kernel state ('/proc/cpuinfo', '/proc/meminfo'), while '/sys' (sysfs) represents the structural hierarchy of physical devices, kernel objects, PCIe busses, and device drivers.

Question 21 | [Hard] [Scenario] — Filesystem Inode & Deletion Semantics

When a user executes 'rm -f large_dataset.bin' on a Linux filesystem, what happens to the underlying storage data blocks if another running application still holds an open file descriptor to that file?

- **(A)** The data blocks are immediately overwritten with zeros, causing the running application to crash with a segmentation fault
- **(B)** The directory entry is unlinked immediately, but the inode and data blocks remain allocated and accessible to the running application until its file descriptor is closed
- **(C)** The command fails with an 'I/O Resource Busy' error and terminates
- **(D)** The file is automatically relocated to the '/tmp/swap' directory

> **Correct Answer**: **(B)** `The directory entry is unlinked immediately, but the inode and data blocks remain allocated and accessible to the running application until its file descriptor is closed`
> **Technical Explanation**: In Linux VFS semantics, 'rm' unlinks the directory pointer (reducing the inode's hard-link count). However, disk blocks are only marked free for reuse when the link count reaches zero AND all open file descriptors referencing that inode are closed.

🔷 Module 4: Shell Scripting

Question 22 | [Easy] [Foundational] — Shell Script Shebang Mechanism

In a Linux Bash automation script, what is the exact function of the initial line '#!/bin/bash' (commonly termed the Shebang)?

- **(A)** It elevates the script's execution privileges to root automatically
- **(B)** It instructs the kernel's 'execve' system call which interpreter binary to launch to parse and execute the script
- **(C)** It compiles the script into an ELF binary executable before running
- **(D)** It marks the entire script file as read-only

> **Correct Answer**: **(B)** `It instructs the kernel's 'execve' system call which interpreter binary to launch to parse and execute the script`
> **Technical Explanation**: The shebang '#!' is a 2-byte magic number (0x23 0x21) recognized by the Linux kernel loader to invoke the specified interpreter binary ('/bin/bash') with the script file path as an argument.

Question 23 | [Easy] [Foundational] — I/O Redirection Operators

Which shell redirection operator appends the Standard Output (stdout) of a command to the end of a specified destination file without overwriting its existing contents?

- **(A)** '>'
- **(B)** '<'
- **(C)** '|'
- **(D)** '>>'

> **Correct Answer**: **(D)** `'>>'`
> **Technical Explanation**: '>>' opens the destination file in append mode (O_APPEND), adding output to the end of the file. '>' truncates and overwrites existing contents.

Question 24 | [Medium] [Numerical] — Bash Arithmetic Expansion Evaluation

In a Bash automation script, what integer result is printed to the terminal when executing the arithmetic evaluation command: 'echo $(((15 % 4) + (2 ** 5) - (12 / 3)))'?

- **(A)** 27
- **(B)** 42
- **(C)** 35
- **(D)** 31

> **Correct Answer**: **(D)** `31`
> **Technical Explanation**: Evaluating step by step according to standard precedence: (15 % 4) = 3; (2 ** 5) = 32; (12 / 3) = 4. Total = 3 + 32 - 4 = 31.

Question 25 | [Medium] [Technical] — Defensive Shell Scripting (set -e)

In production HPC automation scripts, what is the effect of placing the directive 'set -e' at the beginning of the script?

- **(A)** The shell will immediately exit if any command or pipeline returns a non-zero exit status, preventing cascading errors
- **(B)** The shell prints every command to the terminal before executing it
- **(C)** The shell ignores all syntax errors and proceeds execution
- **(D)** The shell allocates maximum GPU memory to all child processes

> **Correct Answer**: **(A)** `The shell will immediately exit if any command or pipeline returns a non-zero exit status, preventing cascading errors`
> **Technical Explanation**: 'set -e' (or 'set -o errexit') ensures defensive execution by causing the shell script to abort immediately when a command fails, preventing silent cascading failures in automated batch workflows.

Question 26 | [Medium] [Numerical] — Linux Process Exit Signal Arithmetic

An HPC batch simulation job terminates unexpectedly. In the shell, checking 'echo $?' yields an exit code of 137. By standard Linux convention, a process killed by an external operating system signal returns '128 + Signal Number'. Which Linux signal terminated the process?

- **(A)** Signal 1 (SIGHUP)
- **(B)** Signal 2 (SIGINT)
- **(C)** Signal 9 (SIGKILL / Out-Of-Memory Killer)
- **(D)** Signal 15 (SIGTERM)

> **Correct Answer**: **(C)** `Signal 9 (SIGKILL / Out-Of-Memory Killer)`
> **Technical Explanation**: Signal number = Exit status - 128 = 137 - 128 = 9. Signal 9 is SIGKILL, the uncatchable termination signal typically issued by the Linux kernel OOM (Out Of Memory) killer when RAM limits are exceeded.

Question 27 | [Hard] [Scenario] — Pipeline Error Trapping Scenario (set -o pipefail)

A student writes an automation pipeline: "cat input.dat | grep 'CRITICAL_METRIC' | awk '{print 2}' > output.txt". The input file 'input.dat' does not exist, yet the bash script continues execution and reports exit status '? = 0'. Why did this silent failure occur, and how should it be prevented?

- **(A)** By default, a shell pipeline returns the exit status of the LAST command in the pipeline ('awk'), masking the failure of 'cat'; enabling 'set -o pipefail' ensures the pipeline returns non-zero if ANY stage fails
- **(B)** 'cat' always returns exit code 0 even if files are missing
- **(C)** Redirection with '>' forces the shell to ignore all previous errors
- **(D)** 'awk' elevated the execution privileges to root

> **Correct Answer**: **(A)** `By default, a shell pipeline returns the exit status of the LAST command in the pipeline ('awk'), masking the failure of 'cat'; enabling 'set -o pipefail' ensures the pipeline returns non-zero if ANY stage fails`
> **Technical Explanation**: Without 'pipefail', a pipeline's exit status is solely that of the last command ('awk'), which succeeded with empty input (status 0). 'set -o pipefail' forces the pipeline to return the status of the rightmost command that failed with a non-zero code.

Question 28 | [Hard] [Technical] — Modern [[...]] vs POSIX [...] Operators

What is an important technical advantage of using the modern Bash compound conditional construct '[[...]]' over the traditional POSIX test operator '[...]'?

- **(A)** '[...]' is executed by the kernel, whereas '[[...]]' runs in user space
- **(B)** '[[...]]' requires compiling the script with GCC before execution
- **(C)** '[...]' only supports floating-point comparisons, while '[[...]]' supports integers
- **(D)** '[[...]]' is a shell keyword that prevents word splitting and pathname expansion on unquoted variable expansions and supports pattern matching with the '=~' regex operator

> **Correct Answer**: **(D)** `'[[ ... ]]' is a shell keyword that prevents word splitting and pathname expansion on unquoted variable expansions and supports pattern matching with the '=~' regex operator`
> **Technical Explanation**: '[[...]]' is built into Bash syntax (a keyword, not a built-in executable like '['/'test'). It does not perform word splitting or glob expansion on unquoted variables, preventing syntax errors when variables are empty or contain spaces, and supports advanced regex matching ('=~').

🔷 Module 5: Secured Shell (SSH)

Question 29 | [Easy] [Foundational] — Standard SSH Port

What is the standard IANA default network TCP port number on which the OpenSSH daemon ('sshd') listens for incoming secure remote shell connections?

- **(A)** Port 80
- **(B)** Port 443
- **(C)** Port 22
- **(D)** Port 8080

> **Correct Answer**: **(C)** `Port 22`
> **Technical Explanation**: Standard IANA port assignment for the Secure Shell (SSH) protocol is TCP port 22.

Question 30 | [Easy] [Foundational] — SSH Public Key Authentication File

When setting up passwordless public-key authentication to an HPC cluster login node, where must the user's PUBLIC key be appended on the remote server?

- **(A)** '~/.ssh/authorized_keys'
- **(B)** '~/.ssh/known_hosts'

- **(C)** '/etc/ssh/ssh_config'
- **(D)** '~/.bashrc'

> **Correct Answer**: **(A)** `'~/.ssh/authorized_keys'`
> **Technical Explanation**: The OpenSSH server verifies incoming client key signatures against the public keys stored in the user's '~/.ssh/authorized_keys' file on the remote machine.

Question 31 | [Medium] [Numerical] — Asymmetric Key Bit Length & Ratio Calculation

Modern HPC clusters recommend 256-bit Ed25519 elliptic-curve SSH keys, which offer equivalent cryptographic security strength to classic 3072-bit RSA keys. By what factor (ratio) is the Ed25519 key size smaller than the equivalent RSA key size?

- **(A)** 4x smaller
- **(B)** 8x smaller
- **(C)** 12x smaller
- **(D)** 24x smaller

> **Correct Answer**: **(C)** `12x smaller`
> **Technical Explanation**: Key size reduction factor = 3072 bits (RSA) / 256 bits (Ed25519) = 12.0x smaller key size, delivering faster handshakes and lower network overhead.

Question 32 | [Medium] [Scenario] — SSH Private Key Permission Troubleshooting Scenario

A user generates an Ed25519 key pair and copies the private key to a new workstation. When running 'ssh -i ~/.ssh/id_ed25519 user@hpc.gla.ac.in', the connection immediately terminates with: 'Permissions 0777 for id_ed25519 are too open'. Which exact command resolves this error?

- **(A)** 'chmod 600 ~/.ssh/id_ed25519'
- **(B)** 'chmod 755 ~/.ssh/id_ed25519'
- **(C)** 'chown root ~/.ssh/id_ed25519'
- **(D)** 'rm -f ~/.ssh/known_hosts'

> **Correct Answer**: **(A)** `'chmod 600 ~/.ssh/id_ed25519'`
> **Technical Explanation**: OpenSSH enforces strict permission checks. If a private key file is readable by group or others (anything more permissive than '600' or '400'), it prints 'WARNING: UNPROTECTED PRIVATE KEY FILE!' and refuses to use the key. 'chmod 600' grants read/write strictly to the owner.

Question 33 | [Medium] [Numerical] — HPC Network File Transfer Time Calculation

A research student transfers a 45 Gigabyte (GB) raw simulation dataset from a local workstation to the cluster storage over a dedicated 1 Gigabit per second (1 Gbps) Ethernet link using 'scp'. Assuming 100% link utilization without protocol overhead, what is the theoretical minimum transfer time in seconds?

- **(A)** 45 seconds
- **(B)** 90 seconds
- **(C)** 180 seconds

- **(D)** 360 seconds

> **Correct Answer**: **(D)** `360 seconds`
> **Technical Explanation**: Total data in bits = 45 GB * 8 bits/Byte = 360 Gigabits (Gb). At a network transfer rate of 1 Gbps: Transfer time = 360 Gb / 1 Gbps = 360 seconds (exactly 6 minutes).

Question 34 | [Hard] [Scenario] — SSH Agent Forwarding Security Risks

What critical security vulnerability is introduced when an administrator or user enables SSH Agent Forwarding ('ssh -A') when connecting to an untrusted intermediate bastion host?

- **(A)** Any user with root privileges on the intermediate host can access the forwarded SSH agent's Unix domain socket and authenticate to other remote servers as you without knowing your passphrase
- **(B)** The intermediate host steals your raw private key file and saves it in '/etc/shadow'
- **(C)** Agent forwarding permanently decrypts all data on your local laptop hard drive
- **(D)** It exposes the client machine to denial-of-service via ICMP flood

> **Correct Answer**: **(A)** `Any user with root privileges on the intermediate host can access the forwarded SSH agent's Unix domain socket and authenticate to other remote servers as you without knowing your passphrase`
> **Technical Explanation**: SSH Agent Forwarding passes a Unix domain socket ('$SSH_AUTH_SOCK') to the remote host. Anyone with root access on that machine can hijack this socket to request signatures from your local key agent, authenticating as you to other machines. 'ProxyJump' ('-J') is the secure alternative.

Question 35 | [Hard] [Technical] — SSH Connection Multiplexing (ControlMaster)

In high-frequency automated HPC scripting environments, what is the architectural benefit of configuring SSH Connection Multiplexing using 'ControlMaster' and 'ControlPath'?

- **(A)** It automatically distributes a single SSH session across multiple physical Ethernet switches
- **(B)** It encrypts SSH traffic twice with independent AES-256 ciphers
- **(C)** It maintains a persistent master TCP socket connection, allowing subsequent SSH/SCP sessions to bypass public-key handshakes and authentication overhead
- **(D)** It prevents compute nodes from communicating with each other

> **Correct Answer**: **(C)** `It maintains a persistent master TCP socket connection, allowing subsequent SSH/SCP sessions to bypass public-key handshakes and authentication overhead`
> **Technical Explanation**: SSH multiplexing ('ControlMaster auto') keeps an initial master connection open. Subsequent 'ssh' or 'scp' invocations communicate through a local control socket ('ControlPath'), reusing the established connection and reducing connection establishment time from ~500ms to ~5ms.

🔷 Module 6: Intel oneAPI Toolkit

Question 36 | [Easy] [Conceptual] — oneAPI Cross-Architecture Vision

What is the primary industry objective of the Intel oneAPI programming model?

- **(A)** To replace the Linux operating system kernel with an Intel-proprietary microkernel
- **(B)** To provide an open, cross-architecture, standards-based unified programming framework that abstracts software development across diverse CPUs, GPUs, FPGAs, and AI accelerators
- **(C)** To restrict high-performance software development exclusively to Intel Core desktop processors
- **(D)** To serve as a commercial subscription-only cloud database engine

> **Correct Answer**: **(B)** `To provide an open, cross-architecture, standards-based unified programming framework that abstracts software development across diverse CPUs, GPUs, FPGAs, and AI accelerators`
> **Technical Explanation**: oneAPI is an open, cross-industry programming model designed to provide a unified development environment across heterogeneous compute architectures (CPUs, GPUs, FPGAs, and specialized AI hardware) without proprietary vendor lock-in.

Question 37 | [Easy] [Foundational] — oneAPI Environment Sourcing (setvars.sh)

Which shell script must be sourced in an HPC Linux environment to set up the necessary PATH, LD_LIBRARY_PATH, and compiler variables for the Intel oneAPI toolkits?

- **(A)** 'source /opt/intel/oneapi/setvars.sh'
- **(B)** 'bash /opt/intel/oneapi/init.sh'
- **(C)** 'exec start_oneapi'
- **(D)** 'python /opt/intel/run_toolkit.py'

> **Correct Answer**: **(A)** `'source /opt/intel/oneapi/setvars.sh'`
> **Technical Explanation**: Sourcing '/opt/intel/oneapi/setvars.sh' initializes all environment variables (compiler paths, library search paths, man pages) required to run oneAPI compilers and libraries in the current shell session.

Question 38 | [Medium] [Technical] — Khronos SYCL C++ Standard

Intel oneAPI DPC++ (Data Parallel C++) is based upon and actively contributes to which open industry standard maintained by the Khronos Group?

- **(A)** CUDA
- **(B)** OpenMP
- **(C)** SYCL (Single-source C++ heterogeneous programming standard)
- **(D)** Vulkan GLSL

> **Correct Answer**: **(C)** `SYCL (Single-source C++ heterogeneous programming standard)`
> **Technical Explanation**: Intel DPC++ is built upon the open Khronos SYCL standard, enabling single-source modern C++ code to execute across diverse accelerators from multiple hardware vendors.

Question 39 | [Medium] [Numerical] — SYCL Work-Group Partitioning Calculation

In a SYCL kernel execution, a 1-dimensional computational domain of global size 2048 work-items is launched with a work-group size specified as 'sycl::range<1>(256)'. How many independent work-groups will the SYCL runtime schedule across the hardware compute units?

- **(A)** 4 work-groups

- **(B)** 8 work-groups
- **(C)** 16 work-groups
- **(D)** 32 work-groups

> **Correct Answer**: **(B)** `8 work-groups`
> **Technical Explanation**: Number of work-groups = Global Range / Local Work-Group Size = 2048 / 256 = 8 work-groups.

Question 40 | [Medium] [Technical] — Unified Shared Memory: malloc_device vs malloc_host

In SYCL Unified Shared Memory (USM), what is the key functional difference between 'malloc_device' and 'malloc_host' memory allocations?

- **(A)** 'malloc_device' allocates memory strictly in accelerator high-bandwidth memory (inaccessible directly by CPU pointers), whereas 'malloc_host' allocates pinned host memory accessible by both CPU and GPU
- **(B)** 'malloc_device' allocates memory on hard disks, while 'malloc_host' allocates memory on tape drives
- **(C)** 'malloc_device' is limited to 1 Megabyte, while 'malloc_host' is unlimited
- **(D)** 'malloc_host' cannot be read by accelerator GPUs

> **Correct Answer**: **(A)** `'malloc_device' allocates memory strictly in accelerator high-bandwidth memory (inaccessible directly by CPU pointers), whereas 'malloc_host' allocates pinned host memory accessible by both CPU and GPU`
> **Technical Explanation**: 'malloc_device' allocates memory resident on the target accelerator device (offering maximum GPU bandwidth, but CPU cannot dereference device pointers). 'malloc_host' allocates pinned host memory accessible by both CPU and device across PCIe.

Question 41 | [Hard] [Numerical] — Unified Shared Memory (USM) Buffer Size Calculation

A developer allocates a 2D square matrix of dimension 2048 x 2048 containing double-precision floating-point numbers ('sizeof(double) = 8 bytes') using 'sycl::malloc_shared'. How many Megabytes (MB) of memory does this single allocation request?

- **(A)** 8 MB
- **(B)** 16 MB
- **(C)** 32 MB
- **(D)** 64 MB

> **Correct Answer**: **(C)** `32 MB`
> **Technical Explanation**: Total bytes = 2048 * 2048 elements * 8 bytes/element = 33,554,432 bytes. In Megabytes: 33,554,432 / (1024 * 1024) = 33,554,432 / 1,048,576 = 32.0 MB.

Question 42 | [Hard] [Scenario] — SYCL In-Order vs Out-of-Order Command Queues

When creating a SYCL command queue, what operational difference exists between an in-order queue ('sycl::property::queue::in_order') and the default out-of-order queue?

- **(A)** An in-order queue can only run on single-core CPUs and cannot target GPUs

- **(B)** Out-of-order queues automatically discard failed execution tasks without notification
- **(C)** In-order queues disable all asynchronous memory transfers
- **(D)** An in-order queue executes submitted command groups strictly in the sequential order they are submitted, whereas default out-of-order queues can execute commands concurrently unless explicit SYCL events/dependencies are declared

> **Correct Answer**: **(D)** `An in-order queue executes submitted command groups strictly in the sequential order they are submitted, whereas default out-of-order queues can execute commands concurrently unless explicit SYCL events/dependencies are declared`
> **Technical Explanation**: By default, SYCL queues are out-of-order, allowing kernels to run concurrently or out of sequence unless directed by event dependencies ('depends_on'). Passing 'sycl::property::queue::in_order' enforces FIFO sequential execution, simplifying debugging.

🔷 Module 7: Intel HPC Toolkit

Question 43 | [Easy] [Foundational] — Intel DPC++/C++ Compiler (icpx)

Which modern LLVM-based C++ compiler provided in the Intel HPC Toolkit is used to compile high-performance SYCL and modern C++ parallel applications?

- **(A)** 'gcc'
- **(B)** 'icpx'
- **(C)** 'javac'
- **(D)** 'cl.exe'

> **Correct Answer**: **(B)** `'icpx'`
> **Technical Explanation**: In the Intel oneAPI HPC Toolkit, 'icpx' is the Intel oneAPI DPC++/C++ compiler, built upon LLVM and providing advanced vectorization and SYCL heterogeneous compilation.

Question 44 | [Easy] [Foundational] — Intel Modern Fortran Compiler (ifx)

Which high-performance Fortran compiler in the Intel HPC Toolkit is the modern LLVM-based replacement for the classic 'ifort' compiler?

- **(A)** 'ifx'
- **(B)** 'gfortran'
- **(C)** 'f90'
- **(D)** 'flang'

> **Correct Answer**: **(A)** `'ifx'`
> **Technical Explanation**: 'ifx' is Intel's modern LLVM-based Fortran compiler, delivering modern Fortran standard support and OpenMP offloading to Intel GPUs.

Question 45 | [Medium] [Technical] — Compiler Architectural Optimization (-xHost)

When compiling computational physics or machine learning C/C++ codes with 'icpx', what optimization does the flag '-xHost' instruct the compiler to perform?

- **(A)** It binds the compiled binary to execute exclusively on a specific network IP address
- **(B)** It runs the code inside an isolated Docker container
- **(C)** It instructs the compiler to generate SIMD vector instructions targeting the highest instruction set supported by the compilation host CPU (e.g., AVX-512)
- **(D)** It disables all loop unrolling and optimization to prioritize compilation speed

> **Correct Answer**: **(C)** `It instructs the compiler to generate SIMD vector instructions targeting the highest instruction set supported by the compilation host CPU (e.g., AVX-512)`
> **Technical Explanation**: '-xHost' queries the CPU where compilation is occurring and instructs the compiler to generate the most advanced SIMD code available (such as AVX-512 or AVX2), maximizing vector arithmetic throughput.

Question 46 | [Medium] [Numerical] — Intel MPI Process Distribution Calculation

An Intel MPI simulation is submitted across 4 cluster compute nodes with the execution command: 'mpirun -n 64 -ppn 16 ./simulation_app'. How many total MPI ranks (processes) are spawned in total, and how many MPI ranks execute per physical node?

- **(A)** 16 total ranks, 4 ranks per node
- **(B)** 32 total ranks, 8 ranks per node
- **(C)** 128 total ranks, 32 ranks per node
- **(D)** 64 total ranks, 16 ranks per node

> **Correct Answer**: **(D)** `64 total ranks, 16 ranks per node`
> **Technical Explanation**: Total ranks = 64 (specified by '-n 64'). Processes per node = 16 (specified by '-ppn 16'). Across 4 nodes: 4 * 16 = 64 total MPI processes.

Question 47 | [Medium] [Scenario] — Intel Inspector Diagnostic Tool

Which diagnostic analysis tool included in the Intel HPC Toolkit is designed to detect dynamic memory corruption, memory leaks, uninitialized memory accesses, and threading data races without needing specialized recompilation?

- **(A)** Intel Inspector
- **(B)** Intel Advisor
- **(C)** Intel Trace Analyzer and Collector (ITAC)
- **(D)** Intel oneDNN

> **Correct Answer**: **(A)** `Intel Inspector`
> **Technical Explanation**: Intel Inspector is a dynamic memory and thread checking tool that identifies subtle memory leaks, dangling pointers, deadlocks, and race conditions in multi-threaded C/C++/Fortran programs.

Question 48 | [Hard] [Scenario] — NUMA Socket Thread Affinity & Memory Allocation Scenario

An HPC engineer profiles an OpenMP simulation on a dual-socket 64-core Intel Xeon node and observes severe memory latency spikes due to cross-socket UPI traffic. Which command pins the process to CPU cores on Socket 0 and restricts physical memory allocation exclusively to Socket 0's local NUMA domain?

- **(A)** Remote NUMA access causes immediate hardware watchdog reboot

- **(B)** numactl --cpunodebind=0 --membind=0 ./simulation_app
- **(C)** nice -n -20 ./simulation_app
- **(D)** nohup ./simulation_app &

> **Correct Answer**: **(B)** `numactl --cpunodebind=0 --membind=0 ./simulation_app`
> **Technical Explanation**: 'numactl --cpunodebind=0 --membind=0' enforces execution strictly on CPU cores belonging to NUMA node 0 and allocates all physical pages from node 0's local memory controller, completely eliminating remote cross-socket UPI bus traversals.

Question 49 | [Hard] [Numerical] — NUMA Memory Access Latency Ratio Calculation

Hardware profiling on a dual-socket Intel Xeon server indicates that local NUMA memory read access on Socket 0 takes 75 nanoseconds (ns), while accessing remote memory attached to Socket 1 across the Intel UPI interconnect takes 135 nanoseconds (ns). What is the percentage increase in memory access latency incurred by the remote NUMA access?

- **(A)** 20% increase
- **(B)** 45% increase
- **(C)** 80% increase
- **(D)** 150% increase

> **Correct Answer**: **(C)** `80% increase`
> **Technical Explanation**: Latency increase = (Remote Latency - Local Latency) / Local Latency = (135 ns - 75 ns) / 75 ns = 60 ns / 75 ns = 0.80 = 80.0% latency penalty.

Question 50 | [Hard] [Technical] — Open Fabrics Interfaces (OFI / Libfabric)

What is the architectural role of the Open Fabrics Interfaces (OFI / Libfabric) provider layer underneath the Intel MPI Library in high-performance computing clusters?

- **(A)** It acts as a modular, hardware-independent network abstraction framework allowing Intel MPI to interface directly with high-speed kernel-bypass fabrics (InfiniBand verbs, RoCE, Omni-Path, TCP) with minimal overhead
- **(B)** It converts MPI communication messages into HTTP REST API queries
- **(C)** It encrypts cluster network packets using software RSA-4096 keys
- **(D)** It assigns dynamic IP addresses to compute nodes using DHCP

> **Correct Answer**: **(A)** `It acts as a modular, hardware-independent network abstraction framework allowing Intel MPI to interface directly with high-speed kernel-bypass fabrics (InfiniBand verbs, RoCE, Omni-Path, TCP) with minimal overhead`
> **Technical Explanation**: Libfabric (OFI) is the core fabric interface in Intel MPI. It provides a standard, low-latency API directly above user-space network drivers (like InfiniBand verbs, Cisco usNIC, Intel PSM3, or sockets), enabling kernel-bypass zero-copy communication across HPC fabrics.

📊 MASTER 50-QUESTION EVALUATION & ANSWER KEY GRID

Q#	Key	Topic	Category	Diff	Q#	Key	Topic	Category	Diff
Q01	**B**	HPC Performance Metrics & TOP500	`Foundational`	`Easy`	**Q26**	**C**	Linux Process Exit Signal Arithmetic	`Numerical`	`Medium`
Q02	**D**	Theoretical Peak Performance (Rpeak) Calculation	`Numerical`	`Medium`	**Q27**	**A**	Pipeline Error Trapping Scenario (set -o pipefail)	`Scenario`	`Hard`
Q03	**B**	The Memory Wall Bottleneck	`Conceptual`	`Medium`	**Q28**	**D**	Modern [[...]] vs POSIX [...] Operators	`Technical`	`Hard`
Q04	**B**	HPC Node Roles: Login vs Compute Nodes	`Scenario`	`Medium`	**Q29**	**C**	Standard SSH Port	`Foundational`	`Easy`
Q05	**B**	Amdahl's Law Speedup Calculation	`Numerical`	`Medium`	**Q30**	**A**	SSH Public Key Authentication File	`Foundational`	`Easy`
Q06	**A**	Gustafson's Law Scaled Speedup Calculation	`Numerical`	`Hard`	**Q31**	**C**	Asymmetric Key Bit Length & Ratio Calculation	`Numerical`	`Medium`
Q07	**A**	Supercomputing Benchmarks: HPL vs HPCG	`Conceptual`	`Hard`	**Q32**	**A**	SSH Private Key Permission Troubleshooting Scenario	`Scenario`	`Medium`
Q08	**C**	Intel AMX Hardware Accelerator	`Foundational`	`Easy`	**Q33**	**D**	HPC Network File Transfer Time Calculation	`Numerical`	`Medium`
Q09	**D**	Intel Data Center GPU Max Microarchitecture	`Foundational`	`Easy`	**Q34**	**A**	SSH Agent Forwarding Security Risks	`Scenario`	`Hard`
Q10	**D**	Habana Gaudi Inter-Accelerator Scale-Out Bandwidth	`Numerical`	`Medium`	**Q35**	**C**	SSH Connection Multiplexing (ControlMaster)	`Technical`	`Hard`
Q11	**C**	Theoretical Peak Memory Bandwidth Calculation	`Numerical`	`Medium`	**Q36**	**B**	oneAPI Cross-Architecture Vision	`Conceptual`	`Easy`
Q12	**D**	AMX 2D Tile Register Capacity Calculation	`Numerical`	`Medium`	**Q37**	**A**	oneAPI Environment Sourcing (setvars.sh)	`Foundational`	`Easy`

Q#	Key	Topic	Category	Diff	Q#	Key	Topic	Category	Diff
Q13	**B**	Intel Xeon CPU Max HBM Operating Modes	`Technical`	`Hard`	**Q38**	**C**	Khronos SYCL C++ Standard	`Technical`	`Medium`
Q14	**A**	Intel AMX vs AVX-512 Throughput Analysis	`Numerical`	`Hard`	**Q39**	**B**	SYCL Work-Group Partitioning Calculation	`Numerical`	`Medium`
Q15	**C**	Linux Unified Filesystem Root	`Foundational`	`Easy`	**Q40**	**A**	Unified Shared Memory: malloc_device vs malloc_host	`Technical`	`Medium`
Q16	**B**	Working Directory Navigation	`Foundational`	`Easy`	**Q41**	**C**	Unified Shared Memory (USM) Buffer Size Calculation	`Numerical`	`Hard`
Q17	**A**	Octal Permission Calculation (chmod)	`Numerical`	`Medium`	**Q42**	**D**	SYCL In-Order vs Out-of-Order Command Queues	`Scenario`	`Hard`
Q18	**D**	Default File & Directory Permissions (umask)	`Numerical`	`Medium`	**Q43**	**B**	Intel DPC++/C++ Compiler (icpx)	`Foundational`	`Easy`
Q19	**B**	Filesystem Block Allocation & Internal Fragmentation	`Numerical`	`Medium`	**Q44**	**A**	Intel Modern Fortran Compiler (ifx)	`Foundational`	`Easy`
Q20	**A**	Virtual Filesystems: /proc vs /sys	`Technical`	`Hard`	**Q45**	**C**	Compiler Architectural Optimization (-xHost)	`Technical`	`Medium`
Q21	**B**	Filesystem Inode & Deletion Semantics	`Scenario`	`Hard`	**Q46**	**D**	Intel MPI Process Distribution Calculation	`Numerical`	`Medium`
Q22	**B**	Shell Script Shebang Mechanism	`Foundational`	`Easy`	**Q47**	**A**	Intel Inspector Diagnostic Tool	`Scenario`	`Medium`
Q23	**D**	I/O Redirection Operators	`Foundational`	`Easy`	**Q48**	**B**	NUMA Socket Thread Affinity & Memory Allocation Scenario	`Scenario`	`Hard`
Q24	**D**	Bash Arithmetic Expansion Evaluation	`Numerical`	`Medium`	**Q49**	**C**	NUMA Memory Access Latency Ratio Calculation	`Numerical`	`Hard`

Q#	Key	Topic	Category	Diff	Q#	Key	Topic	Category	Diff
Q25	**A**	Defensive Shell Scripting (set -e)	`Technical`	`Medium`	**Q50**	**A**	Open Fabrics Interfaces (OFI / Libfabric)	`Technical`	`Hard`